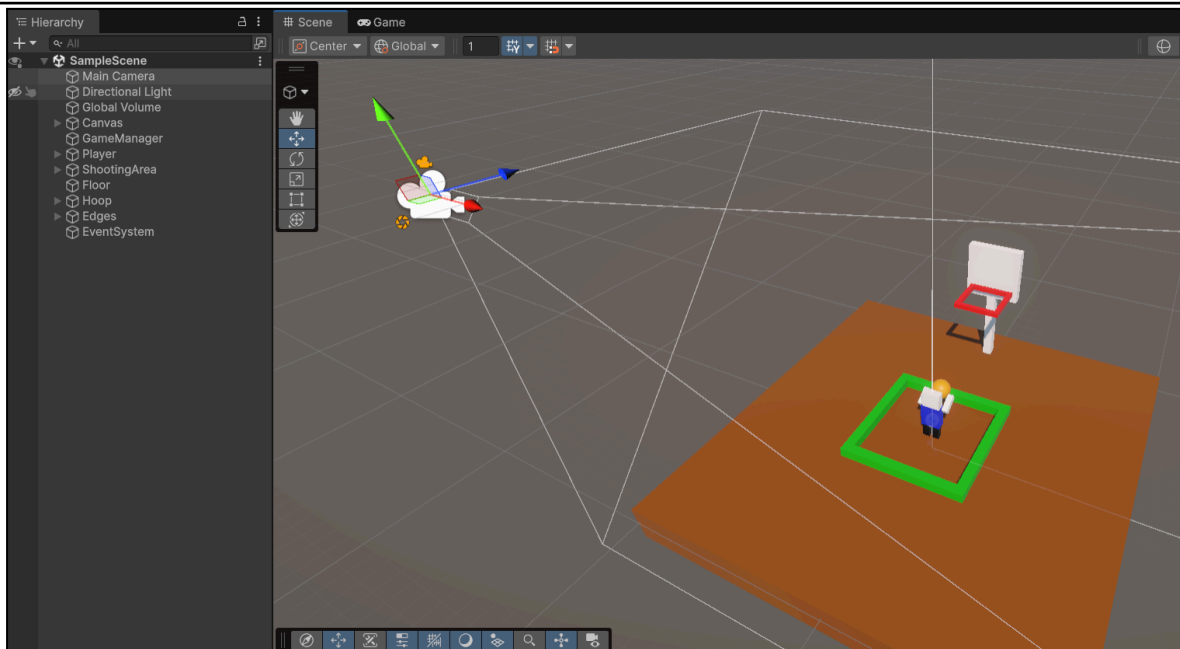


	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 1

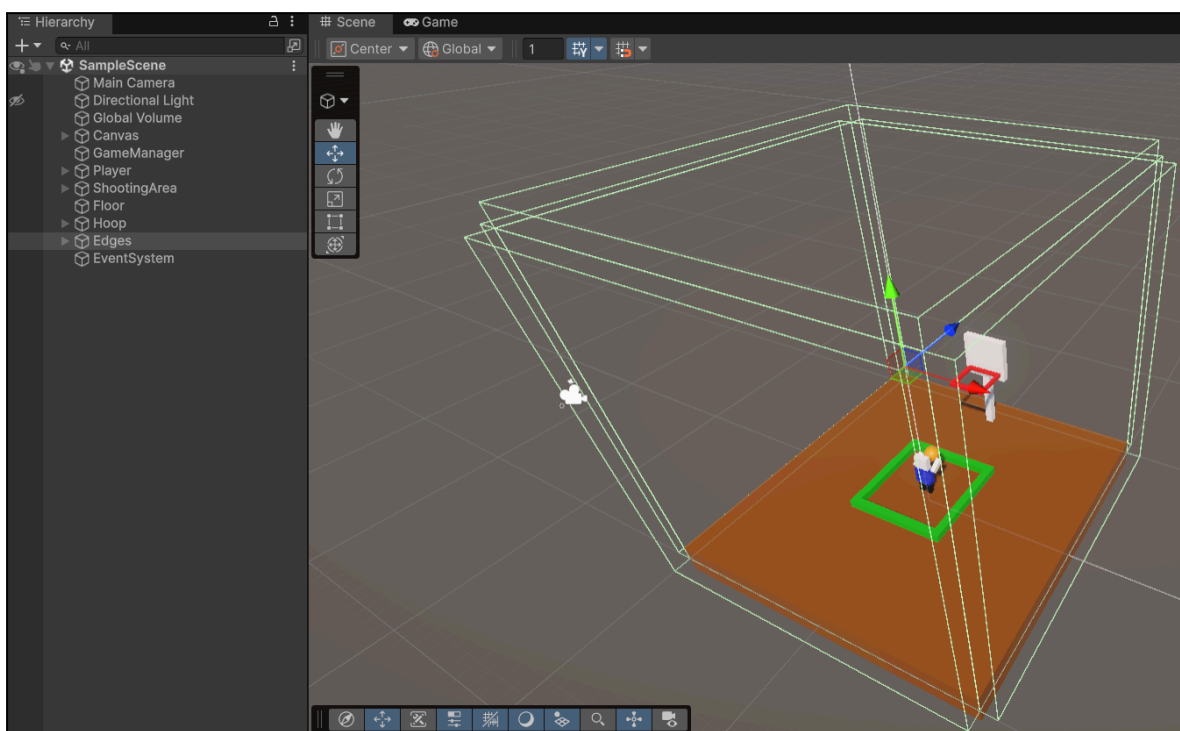
INFORME DE LABORATORIO 01

INFORMACIÓN BÁSICA					
ASIGNATURA:	Computación Gráfica, Visión Computacional y Multimedia				
TÍTULO DE LA PRÁCTICA:	<i>Motores de Física y Colisiones</i>				
NÚMERO DE PRÁCTICA:	01	AÑO LECTIVO:	2026	NRO. SEMESTRE:	IX
FECHA DE PRESENTACIÓN	03/05/26	HORA DE PRESENTACIÓN			
ESTUDIANTE:	Ponce de Leon Aguilar, Marco Antonio			NOTA:	
DOCENTE:	<i>Rene Alonso Nieto Valencia</i>				

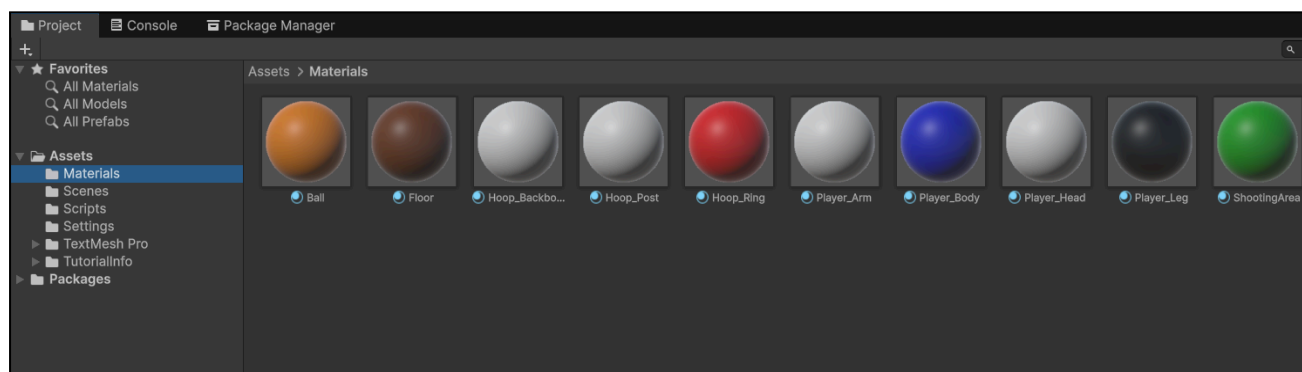
SOLUCIÓN Y RESULTADOS
I. SOLUCIÓN DE EJERCICIOS/PROBLEMAS Para la práctica de laboratorio se desarrolló un videojuego de basketball, en el cual se tendrán que realizar tiros al aro desde diferentes posiciones delimitadas por un área para forzar al jugador a controlar la fuerza del lanzamiento. El objetivo principal es anotar 5 canastas para ganar. 1. Creación del escenario Se utilizaron cubos y esferas que ya nos ofrece Unity y solamente se modificó el tamaño y la posición de estos objetos. El escenario cuenta con los siguientes elementos principales: ● La cámara ● El jugador ● El balón ● El piso ● La zona de lanzamiento ● La canasta



También se tiene un contorno invisible para delimitar el mapa y así el balón no se salga de este, como también el jugador, aunque este internamente ya está delimitado por el área de lanzamiento.

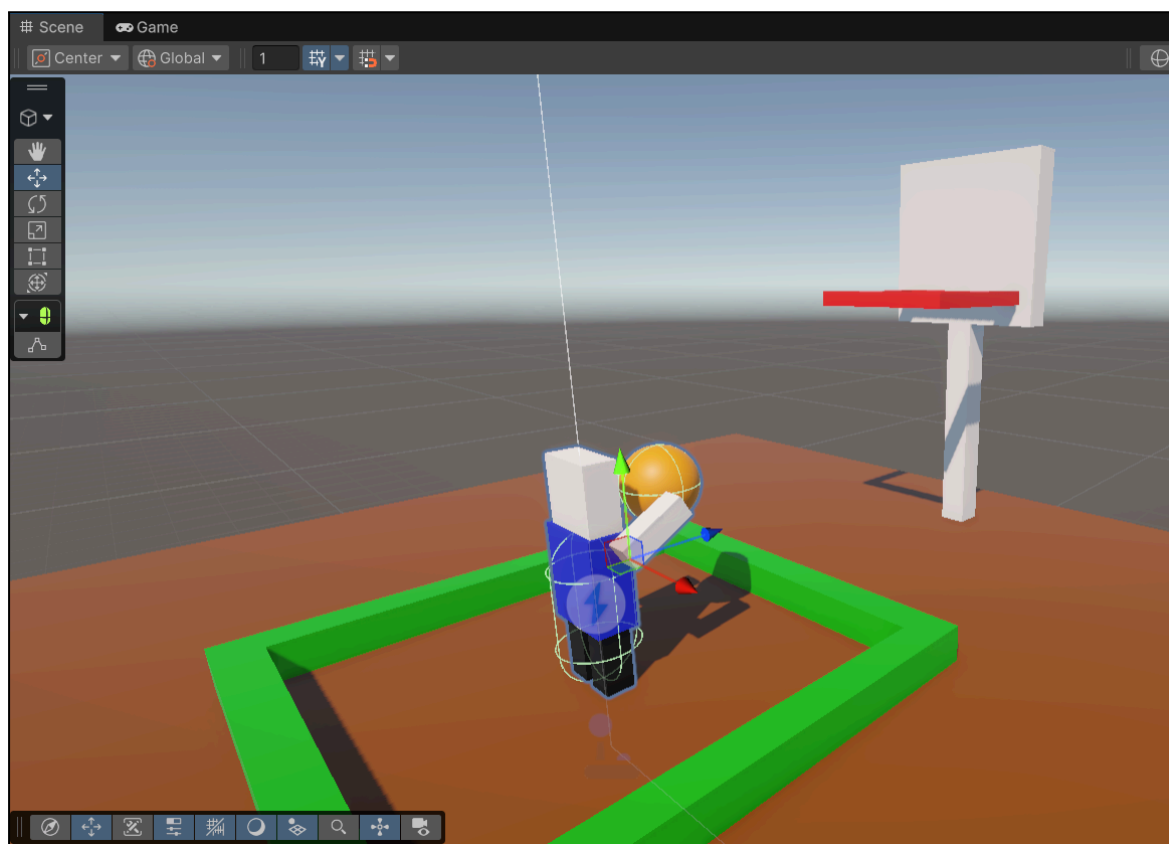


Se crearon algunos materiales para darle color a los diferentes objetos.

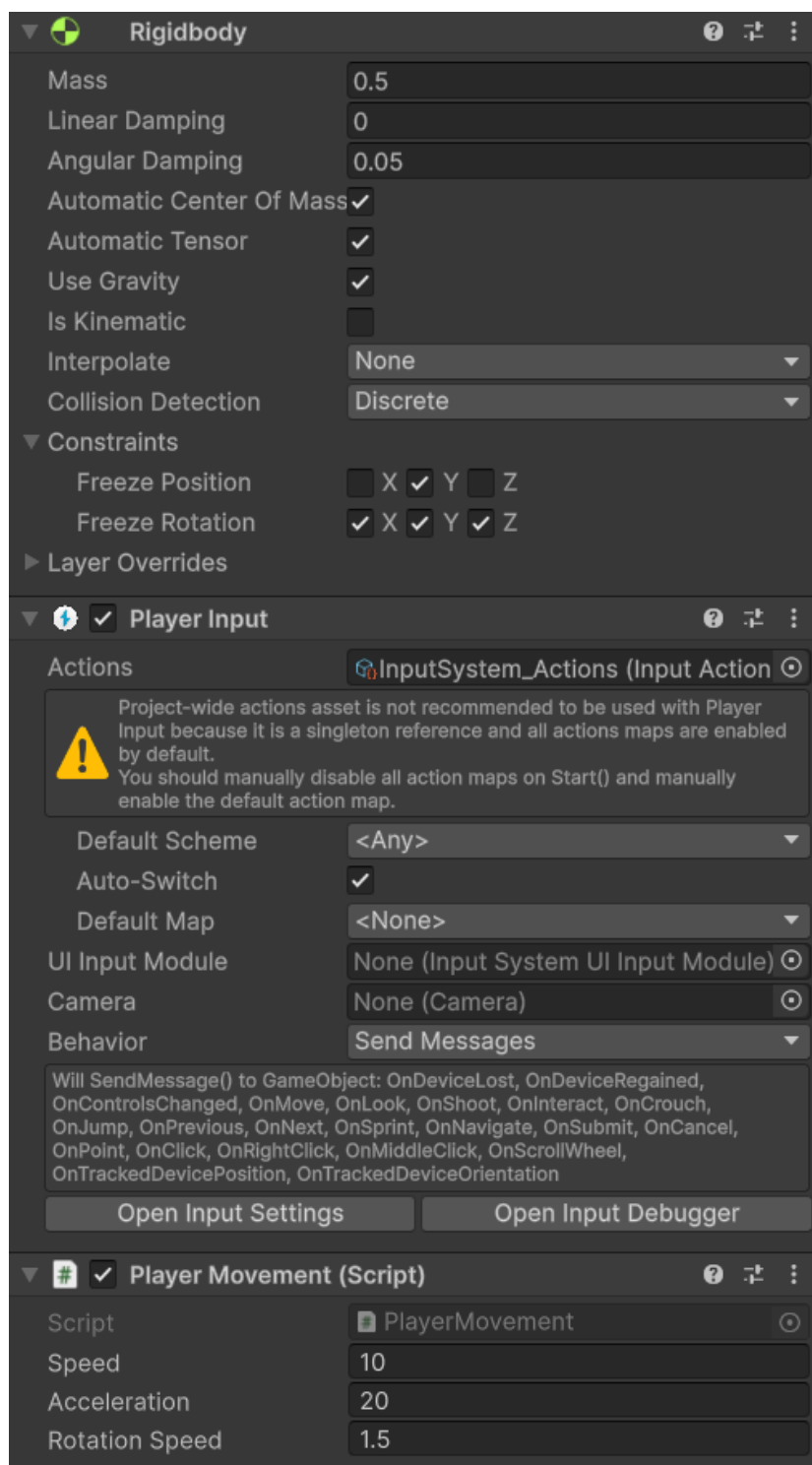


2. Movimiento del personaje

Primero se removieron todos los colliders de los objetos que componen al jugador, y se agregó un collider de cápsula al mismo. Este collider lo mantiene en el piso y colisiona con el área de lanzamiento para no salir de esta zona.



El núcleo para el movimiento del personaje mediante físicas es el componente **Rigidbody**. Además de este componente, también cuenta con un **PlayerInput** y el script **PlayerMovement** con sus respectivas variables.



El código de movimiento es el siguiente:

```
using UnityEngine;
using UnityEngine.InputSystem;

public class PlayerMovement : MonoBehaviour {
    [SerializeField] private float speed = 10.0f;
    [SerializeField] private float acceleration = 20.0f;
    [SerializeField] private float rotationSpeed = 0.15f;

    private Rigidbody rb;
    private PlayerInput playerInput;
    private InputAction moveAction;
    private InputAction lookAction;

    private void Awake() {
        rb = GetComponent<Rigidbody>();
        playerInput = GetComponent<PlayerInput>();
        moveAction = playerInput.actions["Move"];
        lookAction = playerInput.actions["Look"];
    }

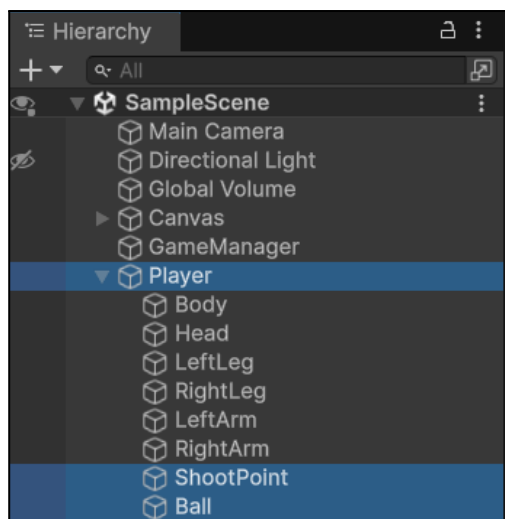
    private void FixedUpdate() {
        if (!playerInput.enabled) return;
        HandleMovement();
        HandleRotation();
    }

    private void HandleMovement() {
        Vector2 input = moveAction.ReadValue<Vector2>();
        Vector3 direction = new Vector3(input.x, 0, input.y).normalized;
        Vector3 targetVelocity = direction * speed;
        targetVelocity.y = rb.linearVelocity.y;
        rb.linearVelocity = Vector3.Lerp(rb.linearVelocity, targetVelocity, acceleration * Time.fixedDeltaTime);
    }

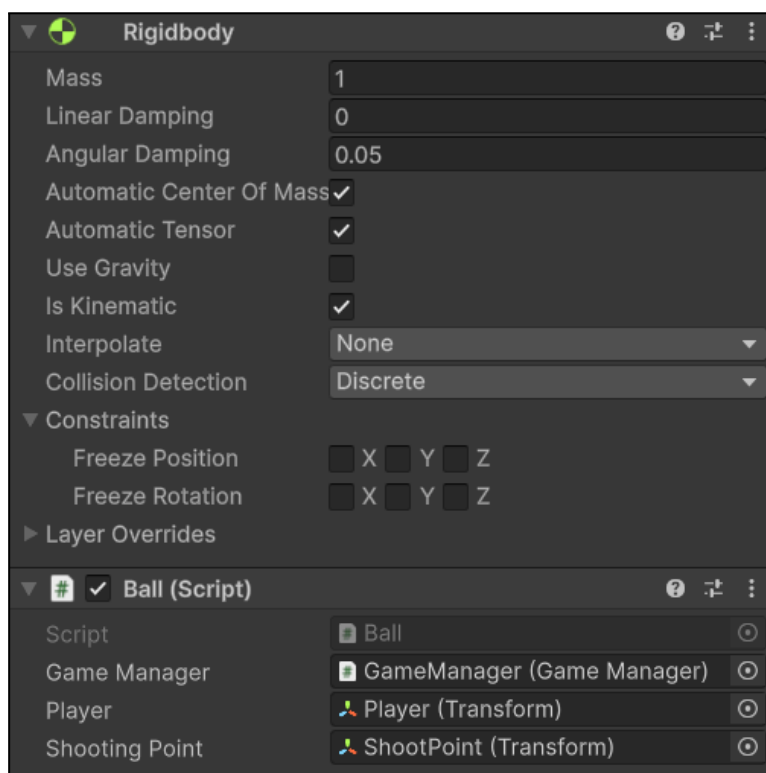
    private void HandleRotation() {
        Vector2 lookDelta = lookAction.ReadValue<Vector2>();
        if (lookDelta.x != 0f) {
            float angle = lookDelta.x * rotationSpeed;
            rb.MoveRotation(rb.rotation * Quaternion.Euler(0f, angle, 0f));
        }
    }
}
```

3. Lanzamiento del balón

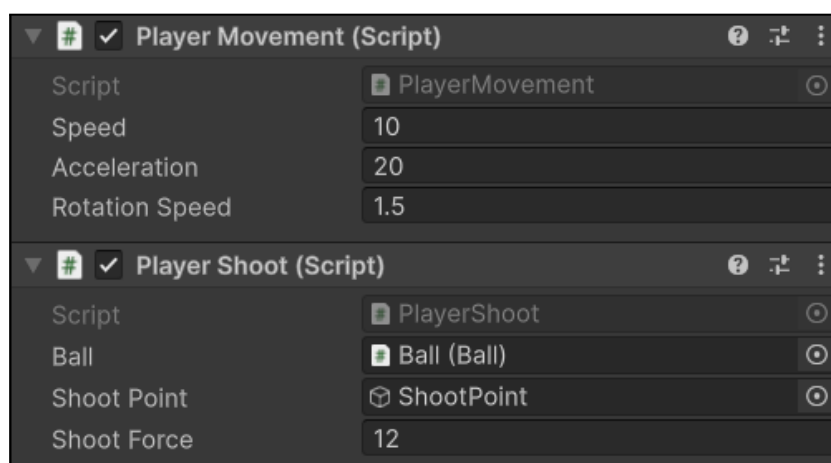
El balón es un objeto hijo del jugador, de tal forma que se moverá con él. También se considera un punto de lanzamiento para reposicionar el balón cada vez que se deba realizar otro intento.



Además de su collider esférico, el balón cuenta con un **Rigidbody** y el script **Ball** para detectar las colisiones posteriormente.



Se crearon las variables respectivas para controlar el lanzamiento del balón, considerando la fuerza y si se tiene el balón en las manos o no, además de las que permiten modificar la fuerza del lanzamiento propiamente. Este script está contenido en el **Player**.



El código que controla el lanzamiento es el siguiente, el cual permite modificar la fuerza del lanzamiento con la rueda del ratón.

```

using UnityEngine;
using UnityEngine.InputSystem;

public class PlayerShoot : MonoBehaviour {
    public Ball ball;
    public GameObject shootPoint;
    [SerializeField] private float shootForce = 12f;

    private float forceStep = 0.25f;
    private float minForce = 8f;
    private float maxForce = 16f;
    private float upwardForce = 2f;
    private PlayerInput playerInput;
    private InputAction shootAction;
    private bool hasBall = true;

    private void Awake() {
        playerInput = GetComponent<PlayerInput>();
        shootAction = playerInput.actions["Shoot"];
    }
}

```

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 8

```

void Update() {
    float scroll = Mouse.current.scroll.ReadValue().y;
    if (scroll != 0) {
        if (scroll > 0) {
            shootForce += forceStep;
        }
        else {
            shootForce -= forceStep;
        }
        shootForce = Mathf.Clamp(shootForce, minForce, maxForce);
        UIManager.Instance.UpdateForce(shootForce);
    }
    if (shootAction.WasPressedThisFrame() && hasBall) {
        Shoot();
    }
}

private void Shoot() {
    hasBall = false;
    ball.transform.parent = null;
    ball.EnablePhysics();

    Vector3 forceDirection = transform.forward + Vector3.up *
upwardFoba;}.Throw(forceDirection, shootForce);
}

public void SetHasBall(bool hasBall) {
    this.hasBall = hasBall;
}
}

```

En el script **Ball** se tienen las funciones para activar y desactivar físicas, además de realizar el lanzamiento propiamente dicho.

```

public void EnablePhysics() {
    rb.isKinematic = false;
    rb.useGravity = true;
}

public void DisablePhysics() {
    rb.isKinematic = true;
    rb.useGravity = false;
}

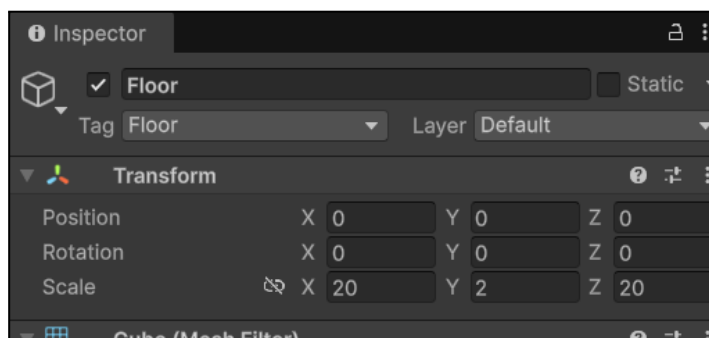
public void Throw(Vector3 forceDirection, float force) {
    rb.AddForce(forceDirection.normalized * force,
ForceMode.Impulse);
}

```

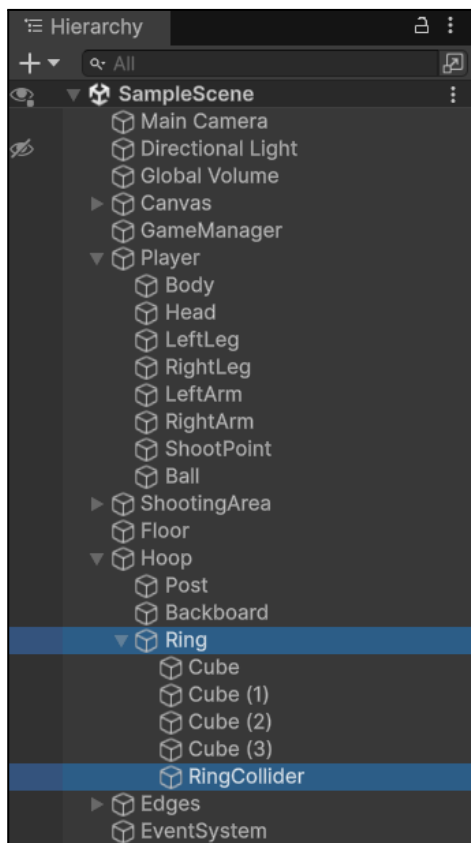

4. Lanzamiento del balón y detección de colisiones

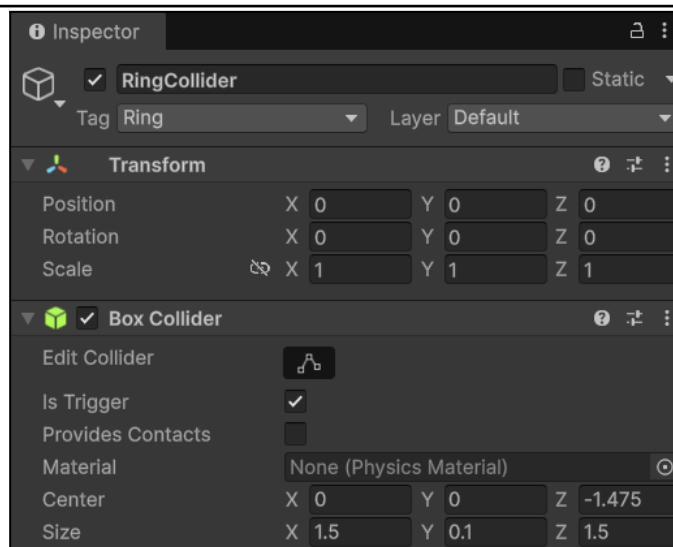
El balón detecta colisión con el piso para regresar a las manos del jugador pasado un tiempo de haber tocado el piso (medio segundo). Como se mencionó previamente, los límites invisibles permiten que el balón no se salga de los límites del mapa.

El piso cuenta con la etiqueta **Floor**.

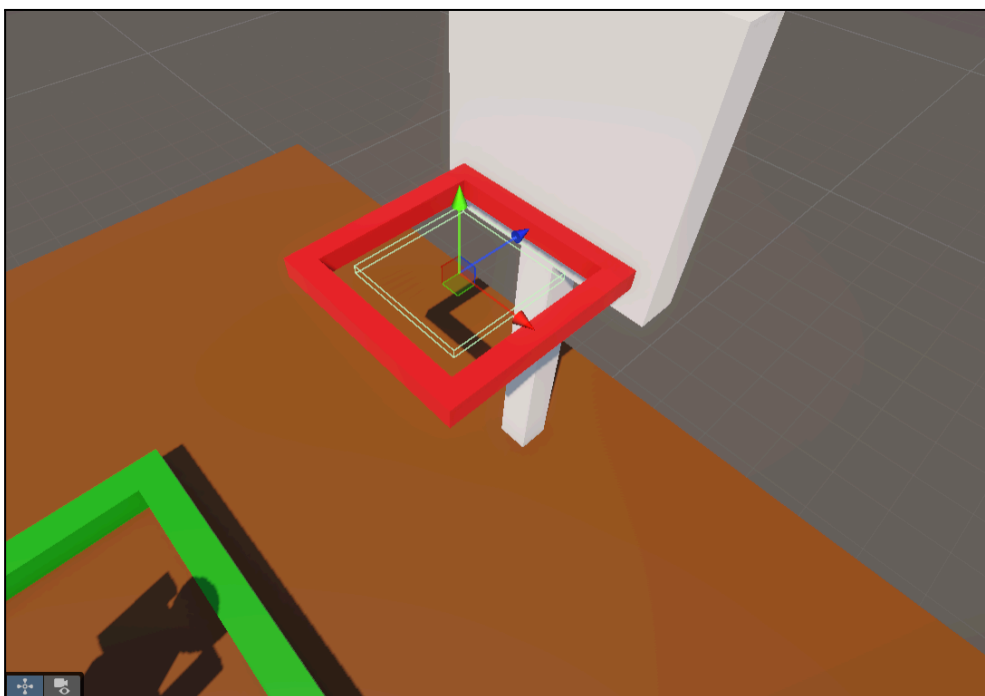


El aro de la canasta cuenta con un collider trigger para no detener la pelota, pero aun así detectar la colisión, y para esto cuenta con la etiqueta **Ring**.





Aquí se puede apreciar la caja de colisión:



El código que se encarga de detectar las colisiones está dentro del script **Ball** mencionado anteriormente. Cuando se colisiona con el piso, el balón retornará al jugador, y al colisionar con la zona de la canasta, se aumentará un punto, esto gestionado por **GameManager** y **UIManager**.

```
using UnityEngine;

public class Ball : MonoBehaviour {
    public GameManager gameManager;
    public Transform player;
    public Transform shootingPoint;

    private const float TIME_TO_RETURN = 0.5f;

    private Rigidbody rb;
    private bool isReturning = false;

    private void Awake() {
        rb = GetComponent<Rigidbody>();
    }

    private void OnCollisionEnter(Collision collision) {
        if(collision.gameObject.CompareTag("Floor") && !isReturning) {
            isReturning = true;
            Invoke(nameof(ReturnToPlayer), TIME_TO_RETURN);
        }
    }

    private void OnTriggerEnter(Collider collider) {
        if(collider.CompareTag("Ring")) {
            gameManager.AddPoint();
            gameManager.SetNewShootingArea();
            Debug.Log("Canasta");
        }
    }

    private void ReturnToPlayer() {
        rb.linearVelocity = Vector3.zero;
        DisablePhysics();
        transform.SetParent(player);
        transform.position = shootingPoint.position;
        player.GetComponent<PlayerShoot>().SetHasBall(true);
        isReturning = false;
    }
}
```

5. GameManager y UIManager

El **GameManager** se encarga de la lógica principal del juego, como modificar aleatoriamente la zona de lanzamiento y reposicionar al jugador. Además, gestiona el puntaje y envía señales a **UIManager** para actualizar el puntaje.

Este script igualmente detecta que se presione la tecla **Esc** para cerrar el juego.

```
using UnityEngine;
using UnityEngine.InputSystem;

public class GameManager : MonoBehaviour {
    private const float SHOOTING_AREA_Y_DEFAULT = 1f;
    private const float PLAYER_Y_DEFAULT = 2f;

    public Transform shootingArea;
    public Transform player;
    public PlayerInput playerInput;

    public const int MAX_POINTS = 5;
    public int points = 0;

    private void Update() {
        if (Keyboard.current.escapeKey.wasPressedThisFrame) {
            Application.Quit();
        }
    }

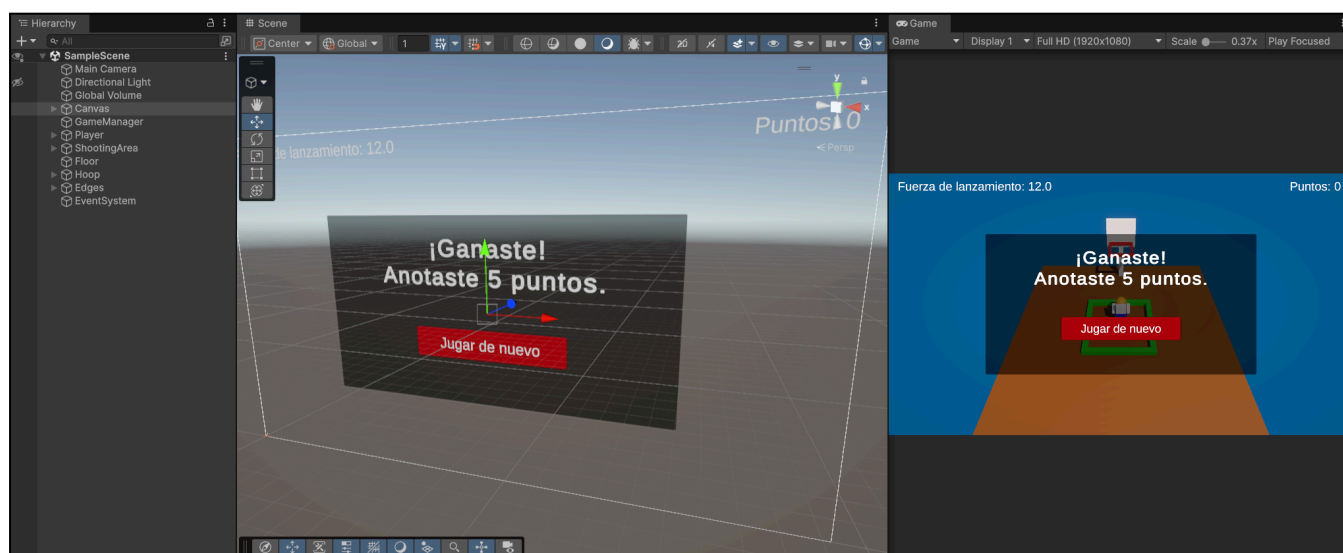
    public void SetNewShootingArea() {
        float newX = Random.Range(-6.5f, 6.5f);
        float newZ = Random.Range(-6.5f, 2.5f);
        shootingArea.position = new Vector3(newX, SHOOTING_AREA_Y_DEFAULT,
newZ); player.position = new Vector3(newX, PLAYER_Y_DEFAULT, newZ);
    }

    public void AddPoint() {
        points++;
        UIManager.Instance.UpdateScore(points);
    }

    public void DisablePlayerInput() {
        Debug.Log("X - deshabilitando input de: " + playerInput.gameObject.name);
        playerInput.enabled = false;
    }
}
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 13

La interfaz de usuario desarrollada es la siguiente, conteniendo las estadísticas como la fuerza de lanzamiento, los puntos realizados, y un panel de victoria que solamente se muestra al anotar los 5 puntos.



El código encargado de modificar estos valores, con las funciones que se llaman desde otros scripts al ser necesario actualizar la interfaz, es un singleton **UIManager** para asegurar consistencia entre escenas, aunque en este proyecto se utiliza únicamente una.

```

using UnityEngine;
using UnityEngine.UI;
using TMPro;
using UnityEngine.SceneManagement;

public class UIManager : MonoBehaviour {
    public static UIManager Instance { get; private set; }

    [SerializeField] private TextMeshProUGUI scoreText;
    [SerializeField] private TextMeshProUGUI forceText;
    [SerializeField] private GameObject winPanel;
    [SerializeField] private Button restartButton;

    private GameManager gameManager;

    private void Awake() {
        if (Instance != null && Instance != this) {
            Destroy(gameObject);
            return;
        }
        Instance = this;
        DontDestroyOnLoad(gameObject);
    }
}

```

```
//Para llamar a la función OnSceneLoaded en lugar de Start
private void OnEnable() {
    SceneManager.sceneLoaded += OnSceneLoaded;
}

private void OnDisable() {
    SceneManager.sceneLoaded -= OnSceneLoaded;
}

private void OnSceneLoaded(Scene scene, LoadSceneMode mode) {
    gameManager = GameObject.FindWithTag("GameManager").GetComponent<GameManager>
();
    winPanel.SetActive(false);
    UpdateScore(0);
    UpdateForce(12);
    restartButton.onClick.RemoveAllListeners();
    restartButton.onClick.AddListener(RestartScene);
}

public void UpdateScore(int points) {
    scoreText.text = $"Puntos: {points}";
    if (points >= GameManager.MAX_POINTS) {
        gameManager.DisablePlayerInput();
        ShowWinPanel();
    }
}

public void UpdateForce(float force) {
    forceText.text = $"Fuerza de lanzamiento: {force}";
}

private void ShowWinPanel() {
    winPanel.SetActive(true);
}

private void RestartScene() {
    SceneManager.LoadScene(SceneManager.GetActiveScene().buildIndex);
}
}
```

6. Enlace al video de demostración

https://drive.google.com/file/d/1OZSCvuMkFRYKN6q8Z7ZtfeinDUzmKB_C/view?usp=sharing

RETROALIMENTACIÓN GENERAL